LUMO: A Multi-agent AI Study Coach

Project Workbook

By
Alshama Mony Sheena
Bhavya Jain
Gautam Santhanu Thampy
Nivedita Nair

10/02/2025

Advisor: Bernardo Flores

TABLE OF CONTENTS

Chapter 1. Literature Search, State of the Art....2
1.1. Literature Search....2
1.1.1. Attention-Aware Learning and Engagement in Elementary Education....2
1.1.2. Mastery-Based Progression and Intelligent Task Ordering in Tutoring Systems....2
1.1.3. Existing AI-Based Tutoring Systems and Their Limitations....3
1.1.4. Towards Multi-Agent Architecture and their benefits....3
1.2. State-of-the-Art Summary....4
Chapter 2. Project Justification....7
Chapter 3. Project Requirements....8
3.1 Requirements....8
3.1.1 Functional Requirements....8
3.1.2 Non-Functional Requirements of Lumo....11
Chapter 4. Dependencies and Deliverables....13
4.1 Dependencies....13
4.2 Deliverables....14
Chapter 5. Project Architecture....16
5.1 Layers and components....16
5.2 Architecture Diagram....18
Chapter 6. Project Design....19
Chapter 7. QA, Performance, Deployment Plan....20
Chapter 8. Implementation Plan and Progress....21
8.1 Programming and Execution Environment....21
8.2 Development Tools for the Team Members....21
8.3 Implementation Flow....22
8.3.1 Phase 1: Environment and Contracts....22
8.3.2 Phase 2: Baseline Reproduction....22
8.3.3 Phase 3: Vertical Workflow for Lesson to Quiz....23
8.3.4 Phase 4: Closed Loop with Feedback and Attention Hooks....23
8.3.5 Phase 5: Analytics, Privacy and Transparency....23
8.3.6 Phase 6: Verification and Validation....23
Chapter 9. Project Schedule....24
Chapter 10. Team Contribution....25

Chapter 1. Literature Search, State of the Art

1.1. Literature Search

1.1.1. Attention-Aware Learning and Engagement in Elementary Education

Attention is vital for learning, along with time spent on a task and sustained focus in challenging topics. Research conducted in elementary level education highlights the importance of short focused lessons and instant feedback as producing strong impact on knowledge retention and mastery [1]. Also, significant contributions have been made by cognitive scientists in assessing and learning attention control in children. Bosch et al. [2] demonstrated that engagement can be quantified in real time through multimodal data in classroom settings, while Whitehill et al. [3] talks about the importance of facial expression analysis in understanding attention orientation. These techniques have measured how flaws in attention control can hinder effective absorption of knowledge, especially in younger children.

Engagement is also vital in adaptive education, and it has been studied using behavioural and multimodal signals, which serve as markers for sustained attention. Eye-gaze dynamics, for example, can identify mind wandering during reading as suggested by Bixler and D'Mello[4]. Existing facial expression recognition models are able to achieve human-level accuracy in identifying student engagement, which enables their use in real-time adaptive technologies [5]. The research conducted in these areas prove that attention and engagement are indeed vital measurable variables that can serve as inputs for intelligent tutoring systems.

1.1.2. Mastery-Based Progression and Intelligent Task Ordering in Tutoring Systems

Mastery-based learning means that the learners have to demonstrate proficiency before they can progress to the next level. Early research has shown that structured mastery based learning improves retention and transfer of knowledge by customizing the material to a student's evolving skill profile [6]. Therefore, knowledge tracing and Bayesian modeling approaches allow tutors to dynamically estimate mastery and assign practice. Piech et al. [7] used Deep Knowledge Tracing (DKT) applied recurrent neural networks to student mastery modeling, which saw improvements in predictive accuracy over traditional Bayesian frameworks. These show how computational models make use of mastery-based progression in adaptive learning environments.

Mastery-based curriculum has been researched in large-scale adaptive platforms. Brusilovsky and Millán [8] reviewed adaptive hypermedia and intelligent tutoring systems and found that ALEKS, a widely used mathematics system, has been shown to produce significant learning gains through individualized mastery-driven progression. Ritter et al. [9] reported significant learning gains from Cognitive Tutor/Carnegie Learning, demonstrating the impact of this method. Anderson et al. [10] proved that Khan

Academy enforces progression that depends on demonstrated competence, with systematic reviews confirming positive impacts on student achievement.

While mastery-based learning is important, intelligent task ordering extends upon the former by considering not only whether a learner has mastered a concept but also which next task will optimize growth. VanLehn [5] framed this distinction as the "inner loop" of step-level guidance and the "outer loop" of problem selection. Collectively, these findings say that mastery-based progression and intelligent sequencing are important in intelligent tutoring systems.

1.1.3. Existing AI-Based Tutoring Systems and Their Limitations

AI-based tutoring systems have now evolved from lab experiments to large-scale deployments. Cognitive Tutor/Carnegie Learning, ALEKS, and ASSISTments are some intelligent platforms that have proven more effective subjects like mathematics, when compared to traditional instruction and have produced gains equivalent to one standard deviation under experimental conditions [11]. These systems suggest that mastery-based progression and adaptive learning in classrooms using AI can go a long way in helping children. Despite the success of the above-mentioned systems, there are limitations and gaps in existing tools.
VanLehn [5] suggests that most of the existing tools rely heavily on the content delivery and assessment, rather than attention and engagement levels of the students. Nkambou et al. [12] elaborates on this by saying that while students benefit from individualized pacing, tutors often fail to sustain motivation, explain beyond correctness, or respond to small indicators of disengagement. Also commercial systems follow a closed-box architecture with no room for transparency in designing the curriculum from a parent/teacher perspective. These constraints highlight the need for next-generation approaches that extend beyond mastery tracking to incorporate engagement analytics, open architecture, and greater pedagogical flexibility.

1.1.4. Towards Multi-Agent Architecture and their benefits

The above mentioned limitations can be overcome by adopting multi-agent architecture in educative platforms. Unlike traditional tutoring systems, multi-agent designs distribute roles such as lesson design, feedback generation, evaluation, and analytics across specialized components that interact cooperatively. Nkambou et al. [12] suggest that modularization improves both scalability and transparency because each agent can be optimized for its pedagogical role, and system behavior becomes easier to explain to educators. Kim et al. [13] have done empirical studies in conversational tutoring which confirm that a combination of AI agents helps in more effective interaction and stronger learning.

Cognitive and motivational theories of learning also support multi-agent architecture. Graesser et al. [14] highlight that effective tutoring requires a "closed learning loop" where explanation, assessment, and motivation are continuously integrated, a principle that directly informs the design of multi-agent educational systems. For example, Murray

et al. [15] emphasize the importance of a system where specialized agents can monitor attention, provide adaptive feedback, and evaluate mastery continuously can be used in helping elementary kids to learn while also looping in teachers/parents. VanLehn [5] and Kim et al. [13] have proven that personalized scaffolding, affect-sensitive feedback, and adaptive curricula can be enabled by agent-based designs. Collectively the literature shows that multi-agent educational environments can mix the strengths of prior ITS with modular, role-based innovations therefore making them well-suited for building next-generation adaptive learning platforms for elementary education.

1.2. State-of-the-Art Summary

Recent advances in adaptive learning provide many leading-edge tools and techniques that move beyond static mastery-based tutoring toward adaptive, personalized and engagement-aware systems. At the architectural level, multi-agent frameworks have emerged as a powerful design strategy. This framework is different as it distributes roles such as lesson sequencing, feedback generation, and engagement monitoring across coordinated agents thus improving transparency and scalability. Empirical studies show that multi-agent conversational tutors increase both the richness of interactions, and student learning outcomes [12][13].

Large language models (LLMs) now power tutoring systems that generate contextualized explanations and adaptive practice questions. LLM-based automatic question generation produces problem sets with configurable difficulty and curriculum alignment, enabling individualized practice that adapts to learner progress [17]. Similarly, prompting methods such as chain-of-thought help with step-by-step reasoning that leads to targeted hints and explanations, offering more than correctness-only feedback [18]. These LLM tools help in designing intelligent systems with real-time adaptiveness.

Engagement analytics is another critical tool. Multimodal signals are being used as indicators that can identify whether a student is focused and attentive. Whitehill et al. [3] showed that facial expression recognition can capture engagement with accuracy comparable to human observers whereas Bosch et al. [2] demonstrated that engagement can be measured in real time through multimodal data in classroom context. Keystroke and gaze-based features further serve as indicators of unattentiveness of a student [4]. These methods extend adaptive tutors with the capability to react to moment-to-moment changes in learner attention, a feature particularly important in elementary education where focus is easily lost.

In a nutshell, the state of the art in educational AI is characterized by multi-agent architectures, LLM-driven personalization and real-time engagement analytics using multimodal signals. These tools and techniques represent the most advanced capabilities available today in adaptive learning research and the design of next-generation systems for elementary learners.

1.3. References

[1] A. Buchem and R. Hamelmann, “The effects of microlearning: A scoping review,” *Educational Technology Research and Development*, vol. 70, pp. 211–240, 2022, doi: 10.1007/s11423-022-10084-1.

[2] N. Bosch, S. D’Mello, R. Baker, J. Ocumpaugh, and V. Shute, “Temporal generalizability of face-based engagement detection,” in *Artificial Intelligence in Education (AIED 2016)*, Lecture Notes in Computer Science, vol. 9684, pp. 38–49, Springer, 2016.

[3] J. Whitehill, Z. Serpell, Y.-C. Lin, A. Foster, and J. R. Movellan, “The faces of engagement: Automatic recognition of student engagement from facial expressions,” *IEEE Transactions on Affective Computing*, vol. 5, no. 1, pp. 86–98, Jan.–Mar. 2014.

[4] R. Bixler and S. K. D’Mello, “Automatic gaze-based user-independent detection of mind wandering during computerized reading,” *User Modeling and User-Adapted Interaction*, vol. 26, no. 1, pp. 33–68, Mar. 2016.

[5] K. VanLehn, “The behavior of tutoring systems,” in *Artificial Intelligence in Education: AI-ED 2007*, Lecture Notes in Computer Science, vol. 4578, pp. 668–675, Springer, 2007.

[6] A. T. Corbett and J. R. Anderson, “Knowledge tracing: Modeling the acquisition of procedural knowledge,” *User Modeling and User-Adapted Interaction*, vol. 4, no. 4, pp. 253–278, Dec. 1995.

[7] C. Piech, J. Bassen, J. Huang, S. Ganguli, M. Sahami, L. J. Guibas, and J. Sohl-Dickstein, “Deep knowledge tracing,” in *Proceedings of the ACM Conference on Learning at Scale (L@S ’15)*, ACM, 2015, pp. 1–10.

[8] P. Brusilovsky and E. Millán, “User models for adaptive hypermedia and adaptive educational systems,” in *The Adaptive Web*, Lecture Notes in Computer Science, vol. 4321, pp. 3–53, Springer, 2007.

[9] K. R. Koedinger, J. R. Anderson, W. H. Hadley, and M. A. Mark, “Intelligent tutoring goes to school in the big city,” in *Proceedings of the 8th World Conference on Artificial Intelligence in Education (AIED)*, Springer, 1997, pp. 421–428.

[10] R. Anderson, A. T. Corbett, K. R. Koedinger, and R. Pelletier, “Cognitive tutors: Lessons learned,” in *Proceedings of the International Conference on Artificial Intelligence in Education (AIED 1995)*, Springer, 1995, pp. 1–19.

[11] K. R. Koedinger, A. T. Corbett, and C. Perfetti, “The role of cognitive tutors in supporting learning: Problem solving and skill acquisition,” in *Technology Enhanced*

Learning: Design Patterns and Pattern Languages, Lecture Notes in Computer Science, vol. 4753, Springer, 2008, pp. 140–165.

[12] R. Nkambou, J. Bourdeau, and R. Mizoguchi, *Advances in Intelligent Tutoring Systems*, Studies in Computational Intelligence, vol. 308, Springer, 2010.

[13] J. Kim, E. Arroyo, K. E. Boyer, and W. L. Johnson, "Multiple agent designs in conversational intelligent tutoring systems," *Technology, Knowledge and Learning*, vol. 24, no. 4, pp. 569–592, 2019.

[14] A. C. Graesser, S. Lu, G. Jackson, H. Mitchell, M. Ventura, A. Olney, and M. Louwerse, "AutoTutor: A tutor with dialogue in natural language," *Behavior Research Methods, Instruments, & Computers*, vol. 36, no. 2, pp. 180–192, May 2004. (Springer)

[15] T. Murray, S. Blessing, and S. Ainsworth, *Authoring Tools for Advanced Technology Learning Environments: Toward Cost-Effective Adaptive, Interactive, and Intelligent Educational Software*, Springer, 2003.

[16] G. Kurdi, J. Leo, B. Parsia, U. Sattler, and S. Al-Emari, "A systematic review of automatic question generation for educational purposes," *International Journal of Artificial Intelligence in Education*, vol. 30, pp. 121–204, Springer, 2020.

[17] I. L. Molina, V. Švábenský, T. Minematsu, L. Chen, F. Okubo, and A. Shimada, "Comparison of Large Language Models for Generating Contextually Relevant Questions" in Technology Enhanced Learning for Inclusive and Equitable Quality Education (EC-TEL 2024), Lecture Notes in Computer Science, vol. 15160. Cham, Switzerland: Springer, 2024, pp. 137–143. doi: 10.1007/978-3-031-72312-4_18.

[18] J. Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," in Advances in Neural Information Processing Systems (NeurIPS), 2022.

Chapter 2. Project Justification

Traditional teaching practices often conform to a "one-size-fits-all" approach, which has a propensity to exclude slower learners while insufficiently challenging more capable students. These flaws cause learners to disengage, gain superficial insights, and yield variable results. Attempts to redress this balance by means of adaptive educative technologies have arisen; however, many are still focused on distributing static content or checking the correctness rather than aiming to address the full cycle including lesson design, practice, assessment, and learner engagement. There is a strong need for systems that can replicate and extend the roles played by educators, tutors, and assessors on a scalable, data-informed basis.

This requirement is met through the suggested LUMO system, which organizes instruction into short, developmentally appropriate lessons, followed then by immediate assessment through quizzes. Concepts are broken down into manageable chunks, so as to alleviate cognitive overload as well as allow preparedness for mastery assessment. A Lesson Designer Agent introduces each concept through contextualized examples that correspond to the learner's personal pace as well as attentional traits. Through the use of misconception-based challenges within quizzes, the Quiz Agent ensures that children cannot simply guess their answers, thereby making mastery measured rather than superficial. The Attention Agent automatically tracks attentional rhythms to recognize the moments the learner is best attuned to receive the next piece of content. It identifies drift or overload, suggests breaks at the appropriate time, and schedules more challenging topics optimally, once attention is most concentrated.

Educators and parents receive a dashboard to monitor not just the content learned, but the dynamics of attention during periods of studying. This degree of awareness takes personalization beyond the content, to the "when" and the "how" of studying. It completes the learning cycle, as well, turning errors into constructive learning experiences. It corrects wrong answers with explanatory clarifiers, suggestive reminders, and motivating prompts to integrate assessment into the teachable process. It, thereby, reinforces the intellectual as well as the emotional aspect of instruction, engendering persistence, long-range proficiency, as well as building confidence.

Together, these innovations validate the project by addressing broadly acknowledged shortcomings within existing tutoring platforms, such as a lack of adequate real-time engagement awareness, inadequate transparency to the instructor, as well as too great a reliance upon accuracy-centered advancement. LUMO extends the practice of educational technology by bringing together attention-aware scheduling, adaptive evaluation, personalized lesson planning, and emotionally nurturing response within a single multi-agent framework. It fulfills the need to provide scalable, personalized, and morally aligned educational assistance for primary schooling.

Chapter 3. Project Requirements

3.1 Requirements

3.1.1 Functional Requirements

Requirement	Priority	User Story	User Purpose	Acceptance Criteria
Lesson Designer – Core Lesson	Essential	When a student starts a new topic, the agent generates a short, curriculum-aligned lesson with visuals/stories.	To receive an engaging introduction to a concept.	Lesson generated quickly; aligned to curriculum;includes at least one story/visual/example.
Lesson Designer – Contextual Explanation	Essential	When a student asks “why” or “how,” the agent provides a contextual explanation in simpler words.	To clarify confusing points.	Agent responds to “why/how” prompt with a clear, age-appropriate explanation; response logged in dashboard.
Lesson Designer – Multi-Format Delivery	Desired	Lessons can be received in multiple formats: text, audio narration, drag-and-drop activity.	To support varied learning styles.	All formats available; students can toggle mode; no content mismatch between formats.
Quiz Agent – Adaptive Difficulty	Essential	After a lesson, student receives a quiz that adjusts difficulty in real time based on performance.	To practice at the right level of challenge.	Quiz difficulty rises if ≥80% correct, drops if ≤40%; difficulty level logged for analysis.

Quiz Agent – Trap-Based Challenge	Desired	For common misconceptions, quiz inserts trap-based distractors (wrong but plausible answers).	To expose and correct misconceptions.	Distractor chosen from error taxonomy; if selected, system triggers Feedback Agent explanation.
Quiz Agent – Summary Chain	Essential	At the end of a unit, user/student receives a summary-chain quiz covering core steps in order.	To reinforce understanding of the whole process.	At least one summary chain per unit; 80% of steps answered correctly before moving ahead.
Quiz Agent – Gamified Mini-Game	Desired	Some quizzes appear as quick mini-games (timed, puzzle format).	To keep assessment fun and motivating.	Game runs smoothly; scoring ties back to mastery tracking; fun mode toggle available.
Feedback Agent – Immediate Correction	Essential	After a wrong answer, the system instantly shows why the answer is incorrect.	To fix errors quickly.	cites correct reasoning; positive tone confirmed by sentiment check.
Feedback Agent – Hint Ladder	Essential	Student can request hints that go from a light cue → guided step → worked example.	To scaffold learning step by step.	At least one hint always available; max 3 hints; hints lead to correct solution in ≤2 retries.
Feedback Agent – Motivation & Nudges	Essential	After multiple errors, the agent sends encouragement (“You’re close!”) or nudges (“Try one more before a break”).	To maintain persistence and discipline.	Encouragement always positive; no punitive tone; logs show reduced quit rate after nudges.

Feedback Agent – Re-Quiz After Support	Essential	After hints or reinforcement (e.g., mini-video), student is re-quizzed on the same concept.	To verify mastery after correction.	Re-quiz delivered automatically; mastery rate ≥80% after re-quiz; progress updates only after success.
Attention Agent – Peak Time Scheduling	Essential	Tracks daily/weekly rhythms; suggests hard topics in high-focus windows.	To learn difficult material at optimal times.	At least 1 peak window logged per day; hard tasks assigned in the window; dashboard shows “focus time” heatmap.
Attention Agent – Drift Detection	Essential	Monitors latency/errors; detects drifting focus and suggests micro-recap or lighter task.	To keep student engaged when losing focus.	Drift flagged within 1 min; recap task triggered; ≥50% of students re-engage (measured by accuracy uplift).
Attention Agent – Break Suggestion	Essential	If overload is detected (error streak/slow times), the agent suggests 1–2 min break with a resume card.	To prevent fatigue and frustration.	Break suggestion per day; resume card accurate;
Attention Agent – Mastery Tracking	Essential	Updates mastery model continuously; links attention patterns to concept learning.	To give teachers/parents deeper insight into progress.	Mastery scores update in the dashboard in real time; report shows attention vs. mastery correlation.
Parent/Teacher Dashboard – Progress	Essential	Parents/teachers log in to view mastery, time spent, strengths/weaknesses.	To monitor learner progress.	Dashboard with accuracy %, time per concept, and strengths/weaknesses clearly.

Adaptive Scheduling	Desired	Lessons auto-rescheduled to align with detected peak focus periods.	To optimize study sessions.	Lesson schedule updates visible; parents/students confirm reordering in the dashboard.
Gamified Micro-Interventions	Desired	Warm-up mini-games before lessons to boost attention.	To prepare learners mentally for new topics.	Game launches successfully
Self-Report Mechanism	Desired	The learner answers quick pings about focus/energy to calibrate the model.	To improve accuracy of attention modeling.	Self-report integrated; correlation visible between self-report and detected attention.
Advanced Attention Modeling	Optional	Opt-in use of camera/microphone for richer signals (processed locally, no raw storage).	To improve detection precision.	Explicit opt-in required;
Federated Learning	Optional	Device models share anonymized updates, not raw data.	To personalize while protecting privacy.	Federated updates run without leaks;

Table 1. List of requirements

3.1.2 Non-Functional Requirements of Lumo

The non-functional requirements safeguard quality, usability and trust in the system.

1. Privacy and Security: The system ensures that no raw audio or video data is ever stored, protecting children's sensitive information. This requirement is critical because the platform will be used by minors, and maintaining confidentiality builds trust with families.

2. Transparency: Every adaptive action taken by the system is accompanied by a clear explanation. For example, if a hint is suggested, the system explains that it is "because a similar mistake was made earlier." This feature is required to help both learners and parents understand the reasoning behind system decisions, increasing confidence and acceptance.

3. Responsiveness: Lessons, quizzes, and feedback must respond quickly to student interactions. Delays break the flow of learning and risk disengagement, so ensuring near-real-time system responses is essential for maintaining engagement.

4. Scalability: The platform should handle many learners and diverse curriculum standards simultaneously. This ensures that the system can grow to accommodate classrooms and schools of varying sizes without performance issues.

5. Usability: The learner interface must remain simple, child-friendly, and engaging, while the parent/teacher dashboard must be intuitive and actionable. This dual design requirement ensures accessibility for both young students and adult supervisors.

Chapter 4. Dependencies and Deliverables

4.1 Dependencies

Dependency	Purpose	Risks / Concerns	Mitigation / Alternatives
On-device analytics & telemetry	Collect and process real-time interaction data (latency, idle time, error rates) for attention modeling	Privacy concerns, device performance issues, potential rejection if using 3rd-party SDKs	Build lightweight in-house analytics; process locally; provide “minimal mode” with limited tracking
Secure storage & user authentication	Protect sensitive student/parent data, manage roles, and enforce access control	Vendor lock-in (e.g., Auth0, Cognito), compliance risks (COPPA, FERPA, GDPR)	Use standards (OAuth2, OIDC); support multiple IdPs; encrypt at rest & in transit; strong consent logs
Curriculum-aligned repositories & educational content	Provide lessons aligned with standards, including licensed images/videos	Paywalls, license restrictions, inconsistent quality	Prefer Open Educational Resources (OER) repositories (CK-12, OpenStax); track licenses; build curated in-house content library
Text-to-Speech (TTS) and visual rendering APIs	Enable multi-modal delivery of lessons (audio narration, visuals)	Pay-per-use APIs can increase costs; latency may disrupt flow	Use on-device TTS where possible; cloud TTS as premium; cache frequently used voices and visuals

Question banks & assessment engine	Deliver validated quizzes with adaptive difficulty and trap-based distractors	Gaps in coverage, reliance on external Open Educational Resources (OER) quality	Combine Open Educational Resources (OER) with educator-authored questions; continuous validation; fallback to fixed difficulty tiers
Feedback & Motivation (LLM integration)	Provide instant hints, motivational nudges, and re-quiz generation	High cost and latency for cloud LLMs; risk of age-inappropriate outputs	Cache scripted hints; small on-device models for common cases; human-in-the-loop calibration
Attention Agent (optional sensors: camera/mic)	Capture richer attention signals (eye gaze, voice activity) for modeling	Major privacy risks, ethical concerns, parent/school resistance	Strictly opt-in; on-device processing only; no raw data storage; always provide a “no-sensor” fallback
Ethical approval & compliance	Enable pilots with minors in schools	Legal/ethical barriers; delays in research approval	Prepare IRB/ethics applications; parental consent workflows; align with child-safety regulations

Table 2. List of dependencies

4.2 Deliverables

The project will produce three primary deliverables.

1. Multi-Agent Learning Application: A functional application integrating four orchestrated agents, the Lesson Designer, Quiz and Challenge, Feedback and Motivation, and Analytics and Attention agents. The tool delivers an adaptive, interactive learning experience for elementary school students. This prototype demonstrates curriculum-aligned lesson generation, adaptive assessments, immediate feedback, and attention modeling.

2. Parent/Teacher Dashboard: A dedicated dashboard agent that consolidates student mastery, time spent, attention patterns, and identifies strengths and weaknesses. This tool equips parents and teachers with evidence-based insights to better support learners and manage schedules effectively.

3. Documentation Report: A comprehensive written report detailing the system design choices, implementation approach, performance evaluation, user feedback, and proposed future enhancements. This document serves as both a technical reference and a roadmap for scaling the system further.

Chapter 5. Project Architecture

The LUMO platform will adopt a multi-agent, layered, service-oriented architecture coordinated by a Planner/Orchestrator that routes learning goals (e.g., *teach fractions*, *quiz addition*) to specialized agents. Functionally, the system closes the learning loop by sequencing micro-lessons, adaptive quizzes, tiered feedback, and attention-aware scheduling, while simultaneously sharing progress and attention trends to parents and teachers through a dashboard. This design addresses the core requirement to show both the functional roles and the technologies we will be using.

5.1 Layers and components

The UI layer (web/mobile) would be implemented with Next.js/React/TypeScript, providing accessible lesson delivery, brief activities, and quiz/game interactions, plus a dedicated Parent/Teacher Dashboard.

The API & Orchestration layer (FastAPI) would handle authentication, sessions, and the central Planner, which would decompose goals and delegate work to agents.

The Agent layer will have four cooperating services: Lesson Designer (renders short, contextual lessons), Quiz & Challenge (adaptive items with misconception-focused distractors), Feedback & Motivation (encouraging nudges, Socratic hints, re-quiz triggers), and Analytics & Attention (continuous mastery and attention modeling).

Shared services include a User Model (mastery and focus profile), a Tool Pool (quiz bank, mini-games, and guarded LLM utilities), and a Policy/Ethics module for guardrails and transparency messaging.

Data & Storage centers on Postgres with content, events, and learner schemas, plus telemetry/analytics logs. Redis/Celery will support async workloads where latency would interrupt flow.

Runtime flow: A typical session begins when a student selects a topic. The Planner requests a concise lesson from the Lesson Designer; once completed, the Quiz Agent serves items at calibrated difficulty with targeted distractors. The Feedback Agent intervenes on errors with tiered hints and can trigger focused re-quizzing, turning mistakes into teachable moments. Throughout, the Analytics & Attention Agent updates

---

[1] TypeScript, *JavaScript with syntax for types*, available at: https://www.typescriptlang.org/
Next.js, *React Framework for Production*, available at: https://nextjs.org/
React, *A JavaScript library for building user interfaces*, available at: https://react.dev/
FastAPI, *Modern, fast (high-performance) web framework for building APIs with Python*, available at: https://fastapi.tiangolo.com/
PostgreSQL, *Open Source Relational Database*, available at: https://www.postgresql.org/
Redis, *In-memory database and cache*, available at: https://redis.io/
Celery, *Distributed task queue*, available at: https://docs.celeryq.dev/

mastery and attention state from interaction signals (e.g., response times and error patterns) and advises the Planner to continue, remediate, or suggest a short break; the Dashboard receives live updates on progress and attention dynamics for adults.

5.2 Architecture Diagram

For reference, the actual diagram can be found at this link.

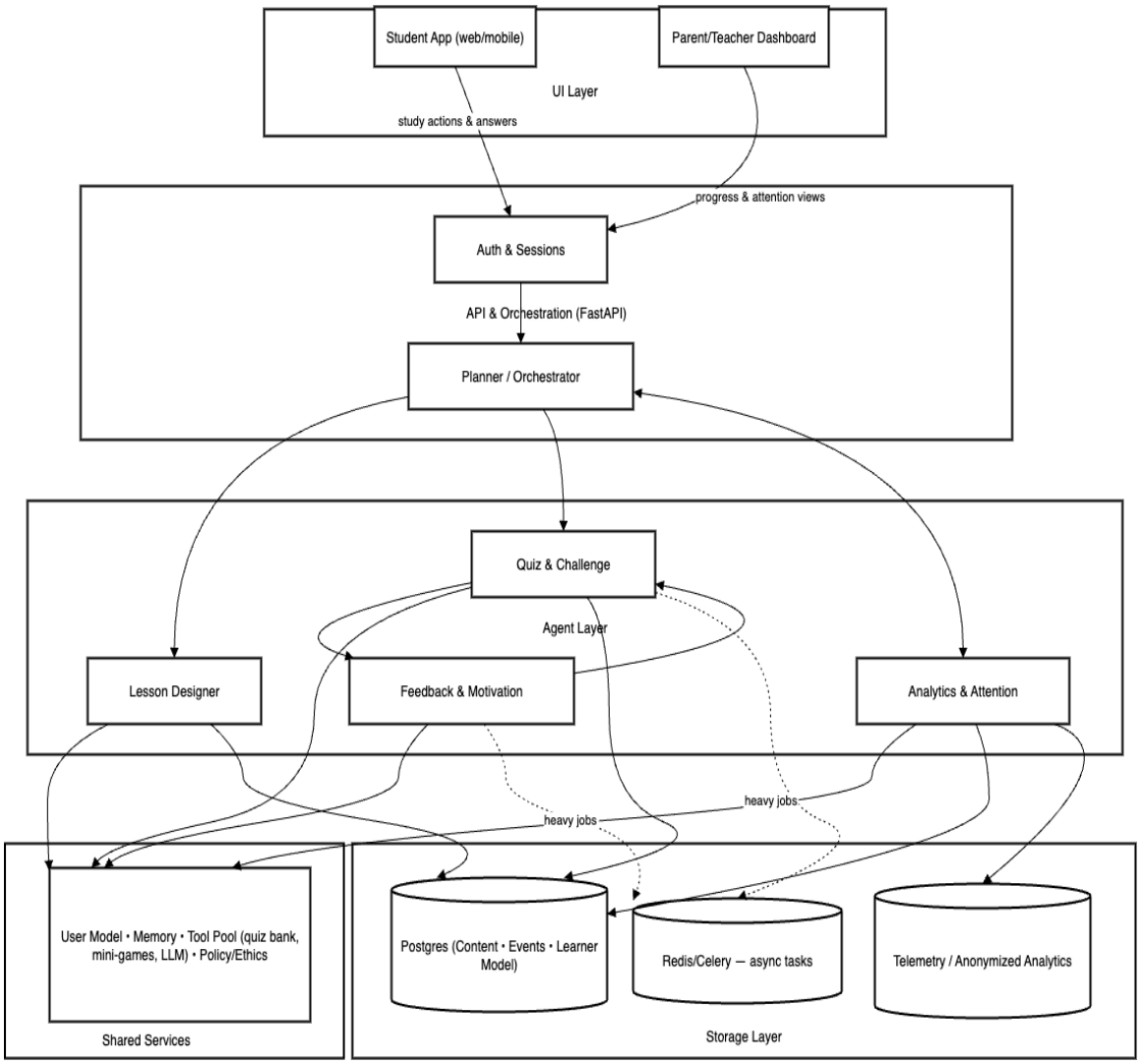

Fig. 1. Proposed architecture diagram

Chapter 6. Project Design

[

Evaluated: Workbook Assignment 2

]

Chapter 7. QA, Performance, Deployment Plan

[

Evaluated: Workbook Assignment 2

]

Chapter 8. Implementation Plan and Progress

The implementation plan for this semester would be to combine research and development to reproduce credible baselines, prototype contributions that push the current SOTA techniques and deliver a demo and technical report by the end of the semester. The work would continue in the next semester with scaled evaluations and deeper modelling and fine tuning.

8.1 Programming and Execution Environment

The first and foremost process would be to set up the environment within our personal devices. The team would be using VSCode on either MacOS/Linux/Windows. To maintain version control we have agreed to use Github repo with three layers or branches, one for the web app consisting of the relevant Next.js/Typescript code. This would be used to maintain the code for the lesson, quiz, feedback interfaces and the dashboard views. The next layer would be a FastAPI service exposing endpoints for content, sessions and analytics. Finally there would be an experimental layer to reproduce current SOTA baseline and for simulations. Environments would be reproducible via *uv*, a python package and project manager and *npm* for the web stack. The team has decided to use a docker compose file to bring up the database for storing analytics results, lesson content locally, allowing the entire stack to run without external services.[2]

8.2 Development Tools for the Team Members

Member	Component	Core Stack	LLM use	Data storage
Gautam	Lesson Design and Generation	React and Next.js (Typescript), MDX authoring, Figma, Storybook	Provides tagged content (e.g., misconception types) to contextualize LLM prompts for other components.	Postgres (Content Schema)

[2]8. Visual Studio Code, *Code Editing. Redefined.*, available at: https://code.visualstudio.com/
9. GitHub, *Where the world builds software*, available at: https://github.com/
10. uv, *Python package and project manager*, available at: https://github.com/astral-sh/uv
11. npm, *Node package manager*, available at: https://www.npmjs.com/
12. Docker Compose, *Define and run multi-container applications*, available at: https://docs.docker.com/compose/
13. MDX, *Markdown for the component era*, available at: https://mdxjs.com/
14. Figma, *Collaborative interface design tool*, available at: https://www.figma.com/
15. Storybook, *Frontend workshop for building UI components*, available at: https://storybook.js.org/

Bhavya	Feedback and Motivation	React (Next.js), FastAPI/Python	Generates tiered, Socratic hints for users based on their specific misconceptions.	Interacts with Redis (task queue) and Postgres for storing/retrieving results.
Alshama	Quiz Generation	FastAPI/Python, Celery	Generates plausible incorrect answers (distractors) for quiz questions.	Postgres (Content Schema)
Nivedita	Analytics and Attention	Python, Postgres	Uses the outputs of user interactions (influenced by LLM feedback) to update the learner model.	Postgres (Events Schema & Learner Model Schema)

Table 3. List of respective development tools

8.3 Implementation Flow

8.3.1 Phase 1: Environment and Contracts

Gautam will set up the Github rep and docker compose file to be used by the team to set up the environment within the Docker runtime, especially the databases, postgres and miniIO. The team will finalize the cross component event schema and API contracts, define privacy guardrails.

8.3.2 Phase 2: Baseline Reproduction

The team will create a minimal, credible baseline on the shared stack. Alshama delivers deterministic quiz heuristics and an API stub integrated with postgres. Gautam implements the minimal lesson renderer with accessibility checks. Bhavya implements a

single level hint loop and wires the re-quiz trigger behind a flag. Nivedita enables event ingestion and computes simple latency metrics and validates dashboards.

8.3.3 Phase 3: Vertical Workflow for Lesson to Quiz

We integrate the renderer with the baseline quiz engine and logging. Nivedita stabilizes logging. Gautam prepares micro lesson packs and finalizes the lesson to quiz handoff contracts. Bhavya connects the feedback placeholder to capture error codes. Alshama exposes quiz endpoints and finalizes the vertical workflow for a single level to quiz generation.

8.3.4 Phase 4: Closed Loop with Feedback and Attention Hooks

We add adaptive hints by error type, re-quiz triggers, and an initial attention score with a simple break scheduler. Gautam adds an example selection policy and short examples strings. Alshama introduces LLM-assisted summary generation gated by validators. Bhavya integrates tiered, LLM drafted hints under guardrails and measures retries-to-mastery and time-to-recovery. Nivedita to ship the initial version of the attention score calculation and a basic scheduler.

8.3.5 Phase 5: Analytics, Privacy and Transparency

Nivedita publishes dashboard views, materialized SQL metrics, and completes the privacy review. Gautam contributes sequencing rules and expands rationale coverage. Alshama implements timed challenges and partial-credit scoring with quality checks. Bhavya adds effective nudges to the workflow.

8.3.6 Phase 6: Verification and Validation

As a team we run unit and integration tests on our components and cross connecting features. This would consolidate the entire prototype. Gautam stabilizes the renderer, conducts usability checks and curates the demo content. Alshama benchmarks the latency and finalizes the quiz bank exports. Bhavya triages any UI bugs and scripts the live walkthrough and user guide. Nivedita to run load tests and combine the results tables/plots which can also be used for the final report.

Chapter 9. Project Schedule

The project schedule lays out LUMO's plan for Fall 2025 across 5 sprints leaving room for buffer for scope creep. The schedule caters to continuous utilization of all the team members with minimal overlap. Milestones are needed at the end of every sprint to ensure that the project is on track.
The complete Gantt chart and task board can be accessed here: [Project schedule link]

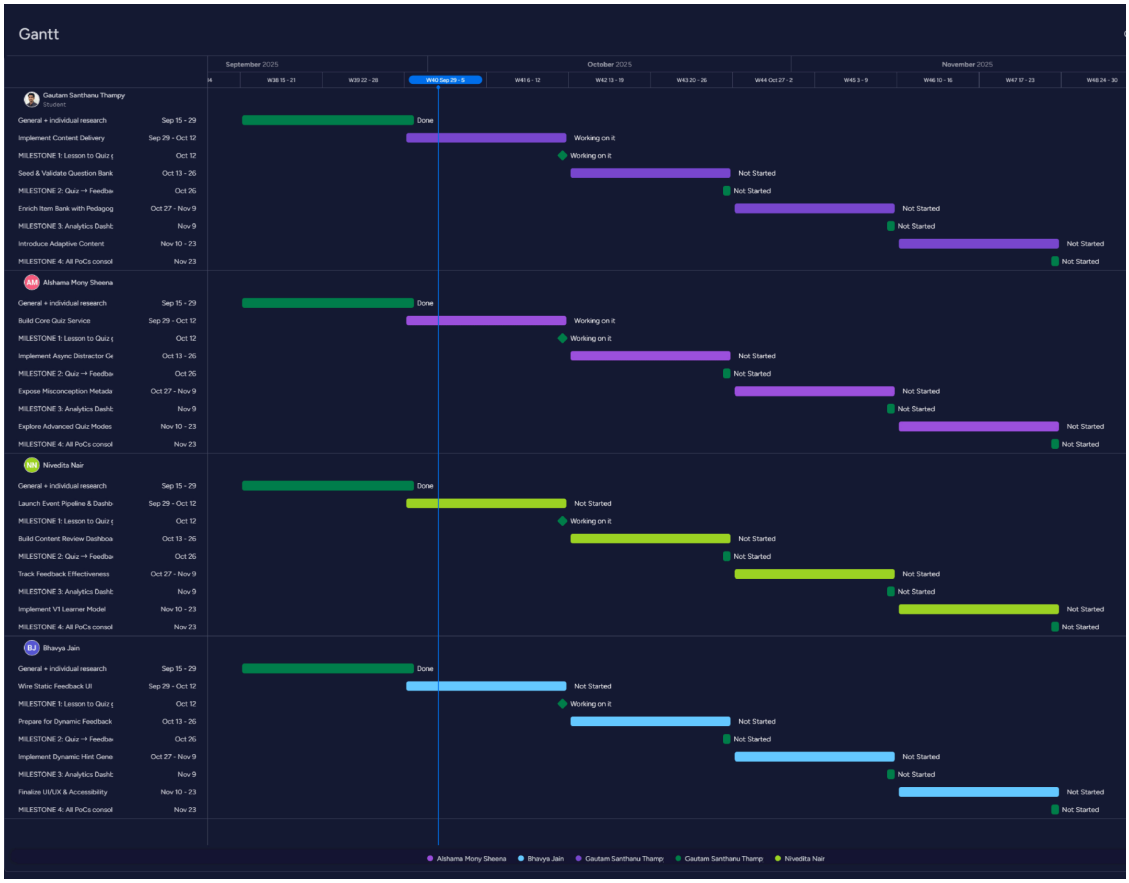

Fig. 2. Quick view of the Gantt chart

Chapter 10. Team Contribution

Team Member	Contribution
Alshama	Literature Search, State-of-the-Art Studies
Bhavya	Project Architecture
Gautam	Implementation Plan and Progress, Project Schedule
Nivedita	Project Requirements, Dependencies and Deliverables